

Documentação do Backend – Casa dos Discípulos

Pré-requisitos

Cada desenvolvedor precisa ter instalado:

1. **Node.js (v16+)**

- `node -v`
- `npm -v`

2. **MySQL** (Server ou via Docker)

- Se não souber usar o terminal MySQL, use **MySQL Workbench** para rodar queries.
- Não é necessário reinstalar bibliotecas; todas estão no package.json comitado.

Configuração Inicial

1. **Clonar o repositório e entrar na pasta**

- `git clone <URL-DO-REPO>`
- `cd backend`

2. **Instalar dependências do projeto**

- `npm install`

3. **Criar/editar arquivo .env**

Crie um arquivo .env com o seguinte template:

DB_HOST=localhost

DB_USER=user

DB_PASSWORD=senha

DB_NAME=CasaDosDiscipulos

DB_PORT=3306

JWT_SECRET=chave

PORT=3001

Importante: .env **não deve ser comitado**. Cada dev deve usar suas credenciais.

4. **Criar o database no MySQL**

Se o usuário do .env não tem permissão para criar DB:

No MySQL Workbench:

- *CREATE DATABASE IF NOT EXISTS CasaDosDiscipulos;*

Ou no terminal:

- *mysql -u root -p*
 - *CREATE DATABASE CasaDosDiscipulos;*
 - *EXIT;*
-

5. Testar conexão com o banco

- *node src/testDB.js*

Saída esperada:

Conexão com o banco OK!

Se der Access denied, revise .env ou crie usuário no MySQL:

- *CREATE USER user1@'localhost' IDENTIFIED BY 'SuaSenha';*
 - *GRANT ALL PRIVILEGES ON CasaDosDiscipulos.* TO user1@'localhost';*
 - *FLUSH PRIVILEGES;*
-

6. Rodar migrations (criação de tabelas)

Opção 1 – via script do package.json:

- *npm run migrate*

Opção 2 – diretamente com Knex:

- *npx knex migrate:latest --knexfile knexfile.js*

Se der erro “No local knex install found”:

- *npm install knex --save*

Para reverter migrations:

- *npx knex migrate:rollback --knexfile knexfile.js*

ou

- *npm run migrate:rollback*
-

7. Rodar o backend em modo desenvolvimento

- *npm run dev*

ou

- `node src/app.js`

Saída esperada:

Servidor rodando na porta 3001

Estrutura do Projeto

src/

```
├── app.js          # Configuração do Express
├── database/
│   └── index.js    # Conexão com MySQL via Knex
├── controllers/    # Validações e chamadas de service
│   └── UsuarioController.js
├── services/       # Regras de negócio e manipulação de dados
│   └── UsuarioService.js
├── models/         # Acesso direto ao banco
│   └── UsuarioModel.js
├── validations/    # Schemas Joi para validar requisições
│   └── usuarioValidation.js
├── middlewares/
│   ├── validate.js # Middleware de validação
│   └── errorHandler.js # Middleware de tratamento de erros
└── utils/
    └── AppError.js # Classe de erros padronizados
```

Padrões de Implementação

1. Controller

- Apenas chama o **service**.
- Não faz manipulação de dados.
- Encaminha erros para o errorHandler via next(err).

2. Service

- Contém regras de negócio.
- Valida duplicidade (ex.: email).
- Aplica hash em senhas.
- Retorna erros esperados via **AppError**.

3. **Model**

- Manipula o banco de dados via Knex.
- Não contém lógica de negócio.
- Retorna null quando não encontrar registros.

4. **Validações**

- Todos os campos devem ser validados com **Joi**.
- `validate(schema)` no middleware antes do controller.

5. **Tratamento de Erros**

- Todos os erros esperados devem usar **AppError**:
- `throw new AppError("Mensagem amigável", statusCode);`
- Erros inesperados retornam 500 sem vaziar dados sensíveis.

6. **Responses**

- Nunca retornar mensagens de SQL ou stack trace.
- Sempre mensagens amigáveis:
- `{ "error": "Email já cadastrado" }`

Boas práticas

1. **Criar novas rotas**

- Rota → Controller → Service → Model.
- Validar entrada com **Joi**.
- Tratar erros com **AppError**.

2. **Evitar lógica no controller**

- Controller deve apenas chamar service e enviar response.

3. **Segurança**

- Nunca retornar SQL ou stack trace para o usuário.

- Senhas sempre criptografadas.

4. Documentação

- Sempre atualizar documentação com novas rotas e validações.
 - Atualizar migrations se houver novas tabelas.
-

Exemplo de Implementação: Gerenciamento de Usuários

Vamos supor que você precise implementar funcionalidades de **gerenciamento de usuários** (CRUD). Siga os passos abaixo para manter **consistência e padrão** com o backend existente.

Passo 1 – Criar Model

- Caminho: src/models/UsuarioModel.js
 - Função: acesso direto ao banco de dados (Knex)
 - Sempre retornar null se não encontrar registros.
-

Passo 2 – Criar Service

- Caminho: src/services/UsuarioService.js
 - Função: regras de negócio, validações de duplicidade, hash de senhas, manipulação de dados.
 - Sempre lançar AppError para erros esperados.
-

Passo 3 – Criar Controller

- Caminho: src/controllers/UsuarioController.js
 - Função: receber requisições, chamar service, retornar response.
 - Não colocar lógica de negócio no controller.
-

Passo 4 – Criar validação

- Caminho: src/validations/usuarioValidation.js
- Função: validar entrada do usuário usando Joi.

- Sempre criar **schemas separados para create e update**.
-

Passo 5 – Criar rota

- Caminho: src/routes/usuarioRoutes.js
 - Função: definir endpoints REST e associar controller + validação.
 - Sempre usar middleware validate(schema) antes do controller.
-

Passo 6 – Testar endpoints

- Use **Postman ou Insomnia**.
 - Sempre testar:
 1. Criação de usuário
 2. Atualização
 3. Busca por ID
 4. Listagem
 5. Exclusão
 6. Validação de erros (ex.: email duplicado, campos inválidos)
-

Passo 7 – Atualizar documentação

- Adicionar collection do postman no github (em artefatos > collections).
- Explicar todos os **erros possíveis**.
- Garantir que a próxima pessoa que implementar siga o mesmo padrão.